

1. 方法多种，关键是找出最优方法（最佳实践、最优解）。
2. 总结、专注、举一反三。
3. 加判断、加循环。一般判断在内，循环在外。
4. while循环 等于 for循环，大于等于 for-in循环。
5. 题目 -> 算法 -> 表达为代码。
6. 算法题目的最优方法往往总是紧贴题目里的假设条件的。去除假设条件后无法作为通用解法。

▪ 回文数

▼ 排序

▼ 冒泡排序、选择排序、插入排序、归并排序、快速排序，它们对应的时间复杂度和空间复杂度各是什么？

- 🇩🇪 五大排序算法复杂度速查表
- 💡 实际怎么选？
- ✅ 修正后的合理顺序（按平均性能）：
- 冒泡排序
- 选择排序
- 自己实现的选择排序（Selection Sort）的变体
- 插入排序
- 归并排序
- 快速排序

▼ 举个简单的、容易理解的递归例子

- 🍌 例子：“问前面的人你是第几排？”
- 🧠 递归过程模拟（假设你在第 4 排）：
- 💡 对应到编程（Python 超简版）：
- 🎯 为什么这个例子好？
- 🧸 再送你一个经典类比：俄罗斯套娃

▼ 查找

- 二分查找
- 在有序数组中插入一个数，保持数组有序。（找到第一个大于等于element的元素，并在此位置插入element）

▼ 哈希表查找

- ✅ 场景 1：只想知道“某个值是否存在” → 用 Set
- ✅ 场景 2：想通过“值”快速找到“索引”（支持重复） → 用 Dictionary
- ✅ 场景 3：只想找“第一次出现的位置” → 用 Dictionary 存首次索引
- 🆚 哈希表 vs 二分查找 vs 线性查找

▼ 交换

- 冒泡操作：比较相邻元素，一轮循环下来，将最大值交换到了最后
- 选择操作：一轮循环下来，将最小值选择（交换）到了最前

▼ 移位

- 插入操作：一轮循环下来，所有元素右移1位。最后一个元素移动到数组头部。

- 一轮循环下来，所有元素左移1位。第一个元素移动到数组尾部。

▼ 反转

- 反转数组

回文数

```
// 回文数
class Solution {
    func isPalindrome(_ s: String) -> Bool {
        let ss: [Character] = Array(s)
        var i = 0
        var j = ss.count - 1

        while i < j {
            let leftChar = ss[i]
            let rightChar = ss[j]

            if !leftChar.isLetter && !leftChar.isNumber {
                i += 1
            } else if !rightChar.isLetter && !rightChar.isNumber {
                j -= 1
            } else {
                if leftChar.lowercased() == rightChar.lowercased() {
                    i += 1
                    j -= 1
                } else {
                    return false
                }
            }
        }

        return true
    }
}

class PBTestOneController: UIViewController, PBTestOneViewDelegate {
    override func viewDidLoad() {
        // MARK: - 测试调用
        let solution = Solution()
        let input = "  1A man, a plan, a canal: Panama1      "
        let result = solution.isPalindrome(input)
        print("输入字符串: \"\((input)\")")
        print("是否为回文串: \"\((result)\")" // 应输出 true
    }
}
```

// 回文数

```
class Solution {
    func isPalindrome(_ s: String) -> Bool {
        let aa: [Character] = Array<Character>(s)

        var tmp: [Character] = []
        for char in aa {
            if char.isLetter ||
                char.isNumber {
                tmp.append(char)
            }
        }
        let ss = tmp

        var i = 0
        var j = ss.count - 1

        while i < j {
            let leftChar = ss[i]
            let rightChar = ss[j]
            if leftChar.lowercased() == rightChar.lowercased() {
                i += 1
                j -= 1
            } else {
                return false
            }
        }

        return true
    }
}

class PBTestOneController: UIViewController, PBTestOneViewDelegate {
    override func viewDidLoad() {
        // MARK: - 测试调用
        let solution = Solution()
        let input = "  1A man, a plan, a canal: Panama1      "
        let result = solution.isPalindrome(input)
        print("输入字符串: \"\\(input)\\\"")
        print("是否为回文串: \"\\(result)\"") // 应输出 true
    }
}
```

排序

冒泡排序、选择排序、插入排序、归并排序、快速排序，它们对应的时间复杂度和空间复杂度各是什么？

 五大排序算法复杂度速查表

排序算法	最好时间	平均时间	最坏时间	空间复杂度	是否稳定
冒泡排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✅ 稳定
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	❌ 不稳定
插入排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✅ 稳定
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	✅ 稳定
快速排序	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$ (平均)	❌ 不稳定

✅ 稳定：相同元素的相对位置排序后不变
❌ 不稳定：可能打乱相同元素的原有顺序

💡 实际怎么选？

- 数据量小 (<50) → 插入排序 (简单又快)
- 要稳定 + 数据量大 → 归并排序
- 追求速度 + 不在乎稳定性 → 快速排序 (记得优化 pivot!)
- 内存极度受限 → 堆排序 ($O(1)$ 空间, 但常数大)
- 泡/选排序 → 除非面试考, 否则别用 😊

✅ 修正后的合理顺序 (按平均性能):

冒泡排序 < 选择排序 < 插入排序 < 归并排序 ≤ 快速排序
(注意: 归并与快排谁在前, 取决于具体实现和数据分布)

冒泡排序

在 Swift 中，冒泡排序（Bubble Sort）是一种简单但效率较低的排序算法，其核心思想是重复遍历数组，比较相邻元素并交换位置，使较大的元素逐渐“冒泡”到末尾。

// 冒泡排序

```
class Solution {  
    // 基础版本（原地排序，使用 inout）  
    func bubbleSortInPlace(_ array: inout [Int]) -> Void {  
        let n = array.count  
        for i in 0..  
n {  
            for j in 0..  
n-1-i {  
                if array[j] > array[j + 1] {  
                    let tmp = array[j + 1]  
                    array[j + 1] = array[j]  
                    array[j] = tmp  
                }  
            }  
        }  
    }  
    /**  
    for i in 0..  
n {  
        for j in 0..  
n-1 {  
            if array[j] > array[j + 1] {  
                let tmp = array[j + 1]  
                array[j + 1] = array[j]  
                array[j] = tmp  
            }  
        }  
    }  
    */  
    /**  
    for i in 0..  
n {  
        for j in 1..  
n {  
            if array[j - 1] > array[j] {  
                let tmp = array[j - 1]  
                array[j - 1] = array[j]  
                array[j] = tmp  
            }  
        }  
    }  
    */  
    /**  
    for i in 0..  
n {  
        for j in 1..  
n-i {  
            if array[j - 1] > array[j] {  
                let tmp = array[j - 1]  
                array[j - 1] = array[j]  
                array[j] = tmp  
            }  
        }  
    }  
}
```

```
        */
    }
}

class PBTestOneController: UIViewController, PBTestOneViewDelegate {
    override func viewDidLoad() {
        // MARK: - 测试调用
        let solution = Solution()
        var array = [1, 8, 8, 9, 4, 6];
        let result: Void = solution.bubbleSortInPlace(&array)
        print(result)
        print(array)
        print(array.sorted())
    }
}
```



```

// 冒泡排序
class Solution {
    // 优化版本（提前终止）
    // 如果某一轮中没有发生任何交换，说明数组已经有序，可以提前结束：
    func bubbleSortInPlace(_ array: inout [Int]) -> Void {
        let n = array.count
        for i in 0..

```

请帮忙动手模拟一下（以 [38, 27, 43, 3, 11, 23] 为例）

[38, 27, 43, 3, 11, 23]

↓

[27, 38, 3, 11, 23, 43]

↓

[27, 3, 11, 23, 38, 43]

↓

[3, 11, 23, 27, 38, 43] ✓

选择排序

选择排序（Selection Sort）是一种简单直观的排序算法。它的基本思想是：每次从未排序的部分中找到最小（或最大）元素，放到已排序部分的末尾。

```

class Solution {
    func selectionSort<T: Comparable>(_ array: [T]) -> [T] {
        var ar = array // 创建一个可变副本

        for i in 0..

```

```

class Solution {
    func selectionSortInPlace<T: Comparable>(_ ar: inout [T]) -> Void {
        for i in 0..

```

请帮忙动手模拟一下（以 [38, 27, 43, 3, 11, 23] 为例）

```

[38, 27, 43, 3, 11, 23]
  ↓
[3, 27, 43, 38, 11, 23]
  ↓
[3, 11, 43, 38, 27, 23]
  ↓
[3, 11, 23, 38, 27, 43]
  ↓
[3, 11, 23, 27, 38, 43] ✅

```

自己实现的选择排序（Selection Sort）的变体

```
class Solution {
    func selectionSort<T: Comparable>(_ ar: [T]) -> [T] {
        var max = ar[0]
        var index = 0

        var objs = [T]()
        var ar2 = Array(ar)

        for _ in 0..
```

插入排序

它的核心思想是：将数组分为“已排序”和“未排序”两部分。初始时，已排序部分只有一个元素（第一个）。算法遍历未排序部分的每一个元素，将其插入到已排序部分的正确位置。

```
class Solution {
    func insertionSort<T: Comparable>(_ array: [T]) -> [T] {
        var ar = array

        for i in 1..
```

```

class Solution {
    func insertionSortInPlace<T: Comparable>(_ array: inout [T]) -> Void {
        for i in 1..

```

```

class PBTestOneController: UIViewController, PBTestOneViewDelegate {
    override func viewDidLoad() {
        // 使用示例
        var unsortedArray = [12, 11, 13, 5, 6]
        print("排序前: \(unsortedArray)")
        Solution().insertionSortInPlace(&unsortedArray)
        print("排序后: \(unsortedArray)")
    }
}

```

请帮忙动手模拟一下（以 [38, 27, 43, 3, 11, 23] 为例）

[38, 27, 43, 3, 11, 23]

↓

[27, 38, 43, 3, 11, 23]

↓

[27, 38, 43, 3, 11, 23]

↓

[3, 27, 38, 43, 11, 23]

↓

[3, 11, 27, 38, 43, 23]

↓

[3, 11, 23, 27, 38, 43] ✓

归并排序

```
// 归并排序主函数
func mergeSort<T: Comparable>(_ array: [T]) -> [T] {
    if array.count <= 1 {
        return array
    }
    /**
    guard array.count > 1 else {
        return array
    }
    */

    let midIndex = array.count / 2
    let leftArray = Array(array[0..
```

```

    }

    return result
}

class PBTestOneController: UIViewController, PBTestOneViewDelegate {
    override func viewDidLoad() {
        // 示例使用
        let numbers = [38, 27, 43, 3, 9, 82, 10, 0, -1]
        let sortedNumbers = mergeSort(numbers)
        print("原数组: \(numbers)")
        print("排序后: \(sortedNumbers)")

        print(sortedNumbers[3...sortedNumbers.count-1], "1")
        print(sortedNumbers[3..

```

请帮忙动手模拟一下（以 [38, 27, 43, 3, 11, 23] 为例）

```

[38, 27, 43, 3, 11, 23]
    ↓ 分
[38, 27, 43]      [3, 11, 23]
    ↓ 分          ↓ 分
[38, 27]  [43]    [3, 11]  [23]
    ↓ 分          ↓ 分
[38] [27]         [3] [11]

    ↑ 合
[27, 38]  [43]    [3, 11]  [23]
    ↑ 合          ↑ 合
[27, 38, 43]      [3, 11, 23]
    ↑ 合
[3, 11, 23, 27, 38, 43] ✅

```

快速排序

```
// 快速排序
class Solution {
    // 实现快速排序
    func quickSort(_ array: [Int]) -> [Int] {
        // 基本情况：如果数组元素少于2个，直接返回
        guard array.count > 1 else {
            return array
        }

        // 选择基准元素（这里选择中间位置的元素）
        let pivot = array[array.count / 2]

        // 分割数组
        let less = array.filter({ (a: Int) -> Bool in
            return a < pivot
        })
        let equal = array.filter() {
            (a: Int) -> Bool in
            return a == pivot
        }
        let greater = array.filter({
            return $0 > pivot
        })

        // 递归排序并合并结果
        return quickSort(less) + equal + quickSort(greater)
    }
}

class PBTestOneController: UIViewController, PBTestOneViewDelegate {
    override func viewDidLoad() {
        // MARK: - 测试调用
        let solution = Solution()
        let array = [1, 8, 8, 9, 4, 6];
        let result: [Int] = solution.quickSort(array)
        print(result)
        print(array)
        print(array.sorted())
    }
}
```

```
// 快速排序
```

```
class Solution {
```

```
    // 原地排序版本 (In-place, 更省内存)
```

```
    // 如果你希望实现原地快速排序以减少内存开销, 可以使用如下版本:
```

```
    func quickSortInPlace<T: Comparable>(_ array: inout [T], low: Int, high: Int) -> Void {  
        if low < high {  
            let pivotIndex = partition(&array, low: low, high: high)  
            quickSortInPlace(&array, low: low, high: pivotIndex - 1)  
            quickSortInPlace(&array, low: pivotIndex + 1, high: high)  
        }  
    }  
}
```

```
// Lomuto 分区方案。单项扫描, 基准在尾。
```

```
// Lomuto 分区方案: 选取最后一个元素作为基准值, 将数组的指定区间 [low, high] 进行重新排列,  
使得小于等于基准值的数都在左边, 大于基准值的数都在右边, 并返回基准值最终应该在的位置索引。
```

```
    func partition<T: Comparable>(_ array: inout [T], low: Int, high: Int) -> Int {  
        // 选择最后一个元素作为基准  
        let pivot = array[high]  
        // 较小元素的索引  
        var i = low - 1  
  
        for j in low..  
            if array[j] <= pivot {  
                i += 1  
                array.swapAt(i, j)  
            }  
        }  
        array.swapAt(i + 1, high)  
  
        return i + 1  
    }  
}
```

```
class PBTestOneController: UIViewController, PBTestOneViewDelegate {
```

```
    override func viewDidLoad() {
```

```
        // MARK: - 测试调用
```

```
        let solution = Solution()
```

```
        var array = [1, 8, 8, 9, 4, 6];
```

```
        let result: Void = solution.quickSortInPlace(&array, low: 0, high:  
array.count - 1)  
        print(result)  
        print(array)  
        print(array.sorted())  
    }
```

```
}  
}
```

请帮忙动手模拟一下（以 [38, 27, 43, 3, 11, 23] 为例）

```
[38, 27, 43, 3, 11, 23]  
  ↓  
[3, 11, 23, 38, 27, 43]  
  ↓  
[3, 11, 23, 38, 27, 43]  
  ↓  
[3, 11, 23, 38, 27, 43]  
  ↓  
[3, 11, 23, 38, 27, 43]  
  ↓  
[3, 11, 23, 38, 27, 43]  
  ↓  
[3, 11, 23, 27, 38, 43] ✓
```

举个简单的、容易理解的递归例子

当然！给你一个**超级简单、生活化、零代码门槛**的递归例子，保证你一听就懂，还能笑着记住 😊

🍌 例子：“问前面的人你是第几排？”

想象你在电影院排队买票，队伍很长。

你不知道自己是第几排（因为没数），但你可以问前面的人！

于是你做了两件事：

1. 如果前面没人了（你是第一个）→ 你就是第 1 排！ ✓ （这是终止条件）
2. 如果前面有人 → 你问他：“你是第几排？” 他回答后，你在他基础上 +1 就行！ 🔄 （这是递归调用）

🧠 递归过程模拟（假设你在第 4 排）：

- 你（第4排）问前面的人：“你是第几排？”
 - 他（第3排）也不知道，就问前面的人：“你是第几排？”
 - 他（第2排）也问前面：“你是第几排？”

- 他（第1排）发现前面没人了 → “我是第1排！”（终止！）
- 第2排听到后：1 + 1 = 第2排
- 第3排听到后：2 + 1 = 第3排
- 你听到后：3 + 1 = 第4排 ✅

这就是递归：自己解决不了，就让“更小规模的自己”去解决，直到问题小到能直接回答。

💡 对应到编程（Python 超简版）：

```
def which_row(you):  
    if you == "first":          # 终止条件  
        return 1  
    else:  
        front_row = which_row(you - 1) # 问前面的人（递归调用）  
        return front_row + 1         # 他在哪排，我就是他+1
```

调用 `which_row(4)` → 返回 4。

🎯 为什么这个例子好？

- 有明确终点（第一排）
- 每一步都在缩小问题（从“我第几排”变成“他第几排”）
- 完全符合递归三要素：
 - i. 终止条件（别无限问下去！）
 - ii. 自我调用（问前面的人）
 - iii. 问题规模变小（离队首越来越近）

🧸 再送你一个经典类比：俄罗斯套娃

- 你拿到一个套娃，想知道里面最小的是第几层。
- 你打开它 → 发现里面还有一个。
- 你继续打开……直到某次打开后没有更小的了 → “啊，这是第5层！”
- 然后一层层合回去，每合一层就 +1。

打开 = 递归调用，最里层 = 终止条件，合上 = 返回结果。

现在你是不是觉得：递归不是魔法，只是“聪明地甩锅给缩小版的自己”？ 😊

如果你想要一个带 **Swift** 代码的小例子（比如算阶乘、数楼梯），或者想看看递归怎么“翻车”（比如忘记写终止条件会怎样），我随时可以展开！你想试试吗？

查找

二分查找

在 Swift 中实现二分查找（Binary Search），前提是数组必须是已排序的（通常为升序）。

```

// 二分查找
class Solution {
    func binarySearch<T: Comparable>(_ array: [T], for target: T) -> Int? {
        var left = 0
        var right = array.count - 1

        // 注意: 这里是 <=, 因为当 left == right 时, 二者相等, 循环可以继续执行一次
        while left <= right {
            let mid = left + (right - left) / 2

            if array[mid] == target {
                return mid
            } else if array[mid] < target {
                // 不 +1 也可以, 无非就是重复判断 mid, 不影响结果
                left = mid + 1
            } else {
                // 不 -1 也可以, 无非就是重复判断 mid, 不影响结果
                right = mid - 1
            }
        }

        return nil
    }
}

class PBTestOneController: UIViewController, PBTestOneViewDelegate {
    override func viewDidLoad() {
        // MARK: - 测试调用
        let solution = Solution()
        let array = [1, 8, 8, 9, 4, 6];
        // 必须先排序
        let result: Int? = solution.binarySearch(array.sorted(), for: 8)
        if let result = result {
            print(result)
        } else {
            print("Not Found")
        }
    }
}

```

请帮忙动手模拟一下（以 [38, 27, 43, 3, 11, 23] 为例，查找 3）


```
[38, 27, 43, 3, 11, 23]
```

```
↓ 必须先排序
```

```
[3, 11, 23, 27, 38, 43]
```

```
↓ mid = 2
```

```
[3, 11]
```

```
↓ mid = 1
```

```
[3]
```

```
↓ mid = 0
```

```
3 
```

在有序数组中插入一个数，保持数组有序。（找到第一个大于等于element的元素，并在此位置插入element）

```
func insertIntoSortedArray<T: Comparable>(_ array: inout [T], _ element: T) {  
    // 使用二分查找确定插入位置  
    var left = 0  
    var right = array.count // 注意: right 初始为 count, 因为我们要在数组末尾插入元素  
  
    while left < right {  
        let mid = left + (right - left) / 2  
        if array[mid] < element {  
            left = mid + 1  
        } else {  
            right = mid  
        }  
    }  
  
    // 在正确位置插入  
    array.insert(element, at: left)  
}
```

```
class PBTestOneController: UIViewController, PBTestOneViewDelegate {  
    override func viewDidLoad() {  
        // 示例用法  
        var numbers = [1, 3, 5, 7, 9]  
        insertIntoSortedArray(&numbers, 6)  
        print(numbers) // 输出: [1, 3, 5, 6, 7, 9]  
  
        var letters: [Character] = ["a", "c", "e", "g"]  
        insertIntoSortedArray(&letters, "d")  
        print(letters) // 输出: ["a", "c", "d", "e", "g"]  
    }  
}
```

哈希表查找

在 Swift 中，**哈希表 (Hash Table)** 的核心实现是通过标准库中的 **Set**（只存储键）和 **Dictionary**（存储键值对）来完成的。它们底层基于哈希表，提供了 **平均 $O(1)$ 时间复杂度** 的查找、插入和删除操作。

下面分别介绍如何用哈希表“查找目标值”：

✅ 场景 1：只想知道“某个值是否存在” → 用 **Set**

如果你有一组**无重复**的元素，并且只关心“目标值是否存在”，使用 **Set** 是最高效的方式。

```
// 构建哈希集合 (O(n))
let numbers: Set = [1, 8, 8, 9, 4, 6] // 自动去重 → {1, 4, 6, 8, 9}

// 查找目标值 (O(1) 平均时间)
let target = 8

if numbers.contains(target) {
    print("找到了 \(target)! ")
} else {
    print("未找到")
}
```

🔍 注意：Set 会自动去重，所以 [1, 8, 8, ...] 只保留一个 8。
如果你需要保留所有重复项并记录位置，就不能用 Set。

✅ 场景 2：想通过“值”快速找到“索引”（支持重复） → 用 **Dictionary**

如果你需要知道目标值出现的所有位置（索引），可以用字典将 值 → 索引列表 映射起来。

```
// 构建哈希表：值 -> [所有出现的索引]
func buildIndexMap(from array: [Int]) -> [Int: [Int]] {
    var map: [Int: [Int]] = [:]
    for (index, value) in array.enumerated() {
        map[value, default: []].append(index)
    }
    return map
}

let array = [1, 8, 8, 9, 4, 6]
let indexMap = buildIndexMap(from: array)

// 查找目标值的所有索引
let target = 8
if let indices = indexMap[target] {
    print("值 \(target) 出现在索引: \(indices)") // 输出: [1, 2]
} else {
    print("未找到")
}
```

输出：

```
值 8 出现在索引: [1, 2]
```

✅ 这样你就既利用了哈希表的 $O(1)$ 查找速度，又保留了原始数组中的所有重复信息和位置。

✅ 场景 3：只想找“第一次出现的位置” → 用 **Dictionary** 存首次索引

```
var firstIndexMap: [Int: Int] = [:]
for (index, value) in array.enumerated() {
    if firstIndexMap[value] == nil { // 只记录第一次
        firstIndexMap[value] = index
    }
}

if let firstIndex = firstIndexMap[8] {
    print("第一次出现在索引 \(firstIndex)") // 输出: 1
}
```

vs 哈希表 vs 二分查找 vs 线性查找

方法	是否需要排序	支持重复	查找时间	额外空间	能否返回索引
哈希表 (Set / Dict)	✗ 不需要	✓ (用 Dict)	O(1) 平均	O(n)	✓ (用 Dict)
二分查找	✓ 必须有序	⚠ 需特殊处理	O(log n)	O(1)	✓
线性查找 (firstIndex(of:))	✗ 不需要	✓	O(n)	O(1)	✓

交换

冒泡操作：比较相邻元素，一轮循环下来，将最大值交换到了最后

```
class Solution {
    func swap(_ array: inout [Int]) -> Void {
        let n = array.count
        for j in 0..
```

选择操作：一轮循环下来，将最小值选择（交换）到了最前

```
class Solution {
    func swap(_ array: inout [Int]) -> Void {
        let n = array.count
        var minIndex = 0
        for j in 1..
```

移位

插入操作：一轮循环下来，所有元素右移1位。最后一个元素移动到数组头部。

```
class Solution {
    func shiftRightInPlace<T: Comparable>(_ array: inout [T]) -> Void {
        let key = array[array.count - 1]
        var j = array.count - 2

        while j >= 0 {
            array[j+1] = array[j]
            j -= 1
        }

        array[j+1] = key
    }
}

class PBTestOneController: UIViewController, PBTestOneViewDelegate {
    override func viewDidLoad() {
        // 使用示例
        var unsortedArray = [12, 11, 13, 5, 6]
        print("排序前: \(unsortedArray)")
        Solution().shiftRightInPlace(&unsortedArray)
        print("排序后: \(unsortedArray)")
    }
}
```

一轮循环下来，所有元素左移1位。第一个元素移动到数组尾部。

```
class Solution {
    func shiftLeftInPlace<T: Comparable>(_ array: inout [T]) -> Void {
        let key = array[0]
        var j = 1

        while j <= array.count-1 {
            array[j-1] = array[j]
            j += 1
        }

        array[j-1] = key
    }
}

class PBTestOneController: UIViewController, PBTestOneViewDelegate {
    override func viewDidLoad() {
        // 使用示例
        var unsortedArray = [12, 11, 13, 5, 6]
        print("排序前: \(unsortedArray)")
        Solution().shiftLeftInPlace(&unsortedArray)
        print("排序后: \(unsortedArray)")
    }
}
```


反转

反转数组

```
class Solution {  
    func reverseArray<T>(_ array: inout [T]) -> Void {  
        var i = 0  
        var j = array.count - 1  
        while i < j {  
            // Swap elements at indices i and j  
            (array[i], array[j]) = (array[j], array[i])  
            i += 1  
            j -= 1  
        }  
    }  
}  
  
class PBTestOneController: UIViewController, PBTestOneViewDelegate {  
    override func viewDidLoad() {  
        // 使用示例  
        var unsortedArray = [12, 11, 13, 5, 6]  
        print("排序前: \(unsortedArray)")  
        Solution().reverseArray(&unsortedArray)  
        print("排序后: \(unsortedArray)")  
    }  
}
```